Review 3 manuscript, October 2026. Code, raw simulation logs and analysis scripts accompany this manuscript (Section VIII).

Detection Rate Is Not Enough: A Simulation-in-the-Loop Framework for Network-Level Evaluation of IoMT Intrusion Detection

MALIKA BALAJI, NEHAA PRABHU, ROHITH PRANOV, SANHIT MUTHUSWAMY, AND RAKSHAN V. P.

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, India

ABSTRACT Dataset-based evaluation cannot expose network-level failure modes. Intrusion detection systems for the Internet of Medical Things (IoMT) are trained and scored on stored captures, where a detector has no queue to slow down, no legitimate device to disconnect, and no feedback from its own decisions. We present a five-step methodology for placing a published IoMT intrusion detector inside a network simulator: obtain the system, audit its reported numbers arithmetically, rebuild it under leakage controls, place its decision rule in the data path of an ns-3 fog network, and evaluate it across 20 seeds, a scenario matrix, a sensitivity analysis and measured inference costs. Applying it to a recent fog-based system and two further IoMT detectors from 2025 and 2026 shows the following. Re-derivation from published confusion matrices confirms a positive-class inversion and transposed precision and recall in one paper and a train/test split drawn after oversampling in two of three; in one of them the test set holds more attack records than the whole dataset. On a public IoMT dataset, oversampling before the split alone raises F1 from 0.673 to 0.959 over 20 partitions, and duplicate rates depend on the feature space retained, from 0.52 % of raw rows to 98.95 % of modelling vectors on Edge-IIoTset. In the network, three threshold rules block every attacker in the baseline yet quarantine between 45 % and 100 % of legitimate wearables, and the published flat rule quarantines all of them in every one of 16 scenarios. A congestion-coupled threshold latches off under load in 20 of 20 baseline seeds and in 11 of 15 further scenarios at the same cost; it also fails at an otherwise safe cost when one fog node carries the flood, and on a Raspberry Pi 4 running the trained ensemble it latches at the Review 2 attack scale even at the board's idle inference cost (3.0 ms per window). Three mitigations each restore full detection in every run, but on the Pi wearable telemetry still waits 0.3 to 1.2 s behind each flood. A trade-off analysis shows that threshold adaptation slides along a curve set by the information it acts on, while gating it with a flow classifier of low false-positive rate moves the curve, cutting false alerts by a third with no missed attacker. We also report that a three-branch ensemble does not beat its best branch, that attack types withheld from training go almost undetected, and that the requested cross-dataset transfer is infeasible because the two datasets share no features. Code, raw runs and analysis are released.

INDEX TERMS Internet of Medical Things, intrusion detection, evaluation methodology, network simulation, ns-3, data leakage, adaptive thresholds, explainable AI, reproducibility.

I. INTRODUCTION

The Internet of Medical Things (IoMT) places networked sensors and actuators directly in the clinical care path. A compromised infusion pump or a silenced cardiac monitor is a patient-safety incident before it is an information-security incident, and this has produced a large body of work on intrusion detection systems (IDS) for IoMT and fog deployments [1, 2, 5, 6, 7, 8, 9].

Almost all of that work is evaluated the same way. A public capture is obtained, class imbalance is corrected, a classifier is trained, and accuracy, precision, recall and F1-score are reported, now routinely above 99 % [1, 2, 5, 9]. **Dataset-based evaluation cannot expose network-level failure modes.** A stored dataset has no queue, so the inference cost of a detector never becomes delay. It has no legitimate devices that can be quarantined, so a rule that disconnects every wearable on a ward scores the same as one that disconnects none, provided both catch the same attackers. And it has no feedback from the detector's decisions to the traffic it later observes, so any failure that lives in that loop is invisible by construction.

This paper presents a methodology for placing a published IoMT IDS inside a network simulator, together with an audit procedure for its reported numbers, and applies both to a recent fog-based system, HIDS-IoMT [1]. We chose that system because it is unusually explicit: it states its threshold ($\tau = 500$ ms), its inspection period, its datasets and hardware, and it publishes the confusion matrices behind its headline metrics. Explicitness is what makes an audit possible, and we treat it as a virtue. To test whether what we find is specific to one paper, we apply the same audit to two further IoMT IDS published in 2025 and 2026 [2, 3], and measure duplicate records and train/test leakage across four public datasets.

Our contributions are:

1. **A reproducible audit methodology.** Five arithmetic checks that a reader can run on a published paper without its code or data (positive-class inversion, transposed precision and recall, partitioning after oversampling, unhandled duplicates, irreproducible metrics), applied to three IoMT IDS papers.

2. **A network simulation framework.** A five-step procedure and an open ns-3 implementation that places a detector's decision rule in the data path of a fog network, measures what dataset evaluation cannot (collateral quarantine of legitimate devices, delay including inspection queueing, control overhead, inspection energy), and evaluates it over 20 seeds, a one-factor-at-a-time scenario matrix, a sensitivity analysis and measured inference costs.

3. **Empirical findings, including negative results.** In the baseline, detection rate is identical across three threshold architectures while collateral quarantine ranges from 45 % to 100 % of the wearable estate, and the published flat rule quarantines every wearable in all 16 scenarios; a congestion-coupled threshold latches off in 20 of 20 baseline seeds under load and in 11 of 15 further scenarios at the same cost, and three mitigations each restore full detection in every run; threshold adaptation alone moves operating points along a fixed trade-off curve, while gating it with a flow classifier moves the curve; oversampling before partitioning inflates F1 from 0.673 to 0.959 on an IoMT dataset; and duplicate rates depend on the feature space, from 0.52 % of raw rows to 98.95 % of modelling vectors on Edge-IIoTset. We also report that our deep ensemble does not beat its best branch, that one explanation method

is unstable, and that the detector does not recognise attack types withheld from training.

Section II positions the work. Section III formalises the framework, and Section IV describes how it was applied. Section V reports results, and Sections VI to VIII discuss, bound and conclude.

II. RELATED WORK

A. IOMT INTRUSION DETECTION

Recent IoMT detectors span convolutional and recurrent hybrids [1, 5], gradient-boosted trees with post-hoc explanation [2], federated training across clinical sites [6], meta-learned ensembles [7], feature-selection pipelines [8], lightweight attention networks [9] and CNN classifiers on the CICIoMT2024 benchmark [10, 11]. They are evaluated on public captures, principally WUSTL-EHMS-2020 [13], IoTID20 [14], Edge-IIoTset [15] and CICIoMT2024 [11], and they report accuracy at or above 97 %, most above 99 %. HIDS-IoMT [1] adds fog deployment on a Raspberry Pi and is the only one of these to state a decision threshold and an inspection period, which is why it is the object of our network-level study. Its prior-work neighbour in our own laboratory, a stacking ensemble on CICIDS2017 [12], applies its balancing step before the train/test split, the same ordering we examine in Section V-A. When most of a literature reports figures within a point of the ceiling, the interesting question becomes whether the evaluation instrument still resolves anything.

B. EVALUATION PITFALLS AND DATASET FLAWS

Arp *et al.* [17] catalogue pitfalls in learning-based security, including data snooping and inappropriate baselines, and show them to be common in top-venue work. For network IDS specifically, Engelen *et al.* [18] and Lanvin *et al.* [19] document generation, flow-construction and labelling errors in CICIDS2017 that change benchmark conclusions, Liu *et al.* [20] extend this to CSE-CIC-IDS2018, and Flood *et al.* [21] define measurable "design smells" across seven benchmarks. Galal [23], concurrently with our project's own finding, reports that missing values in Edge-IIoTset are serialised differently in normal and attack files and leak the label.

Bouke and Abdullah [22] study how pattern leakage introduced during preprocessing affects the reliability of machine-learning IDS on benchmark datasets. Our work starts from that concern and extends it in three directions. First, we treat exact duplicate records as a leakage vector that survives a correct split order and show that its size is a property of the feature space a pipeline keeps, not of the file. Second, we audit published confusion matrices arithmetically, which exposes partitions drawn from an oversampled population without any access to code. Third, we move evaluation from the dataset to the network, where failure modes such as false quarantine of legitimate devices and a latch-off of congestion-coupled thresholds appear that no dataset can show.

C. NETWORK SIMULATION FOR DETECTOR EVALUATION

Simulation is used to evaluate detectors in other domains: Meyer *et al.* [25] build an OMNeT++ toolchain that injects anomalies into time-sensitive automotive networks and show that detection quality depends on the traffic mix. In IoT security, ns-3 is mostly used to generate data for offline classifiers [27] or to measure the impact of attacks on network performance [28], and recent testbeds emulate mixed traffic for live detection [26]. We found no IoMT work that places a published detector's decision rule in the forwarding path and measures what it does to legitimate devices; that is the gap this paper addresses.

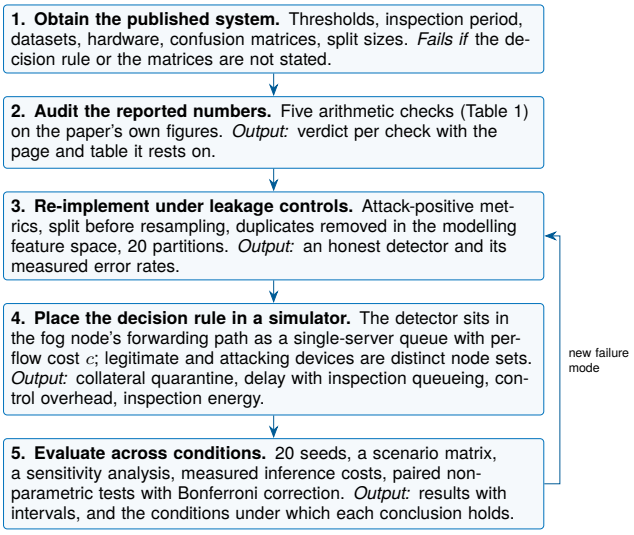


Figure 1. The five-step framework. The feedback arrow is how mitigations enter: a failure found in step 5 is fixed in the rule and re-evaluated, as for the latch-off in Section V-F.

D. ADAPTIVE THRESHOLDS

Dynamic thresholds that follow recent error statistics are well established in anomaly detection [29, 30, 31]. They adapt to the detector's own error distribution. The congestion-coupled rule we study instead adapts to the network's queue state, and Section V-F shows why that coupling is dangerous when the detector sits in the data path.

III. A FRAMEWORK FOR NETWORK-LEVEL IOMT IDS EVALUATION

The framework has five steps (Fig. 1). Each produces an artefact that the next consumes, and each has an explicit failure condition, so a step that cannot be completed is reported as such rather than worked around.

A. STEP 2: THE AUDIT CHECKS

The checks need only quantities a paper prints (Table 1). Check (a) recomputes precision and recall from the confusion matrix with each class as positive and asks which convention reproduces the printed figures. Check (b) asks whether the printed precision equals the recomputed recall and vice versa, or whether the printed formulae are swapped. Check (c) sums the reported split sizes and compares them with the dataset sizes before and after oversampling; a test fold that matches the oversampled total, or that contains more minority samples than the raw data holds, proves the partition was drawn after oversampling. Check (d) searches the text for duplicate handling and, where the data is available, measures duplicates directly. Check (e) records any printed metric that cannot be reproduced from the matrix.

B. STEP 4: THE DETECTOR IN THE DATA PATH

A fog node with per-flow inspection cost c serves at most $S = 1/c$ flows per second. Its queue occupancy $L \in [0, 1]$ (queue length over capacity) is the load signal an adaptive rule may read. Each inspected flow updates a per-source record of inter-

Table 1. Audit checks. Each needs only the paper's printed numbers.

	Check	Evidence that confirms it
a	Positive-class inversion	printed P/R reproduce only with Normal as positive
b	Transposed precision/recall	printed P equals recomputed R, or formulae swapped
c	Partition after oversampling	split sizes sum to the oversampled total; test minority exceeds raw minority
d	Duplicates unhandled	no mention, and duplicates present in the data
e	Irreproducible metric	a printed value not derivable from the matrix

arrival gaps, and a source accumulating p violations of the rule (the *patience*) is quarantined: its later packets are dropped at the fog node and a quarantine notice is sent to the controller and to every peer fog node. A quarantined legitimate device is a false positive at the network level, and it is counted separately from false-positive alerts.

Two measurements matter that a flow monitor on the wire does not provide. The first is application-level delay: each packet carries its send time, and delay is measured when the fog node finishes inspecting it, so inspection queueing is included. The second is inspection energy, $E = NP_{\text{idle}}T + (P_{\text{active}} - P_{\text{idle}})\, n_{\text{insp}}\, c$ for N fog nodes over T seconds and n_{insp} inspections. The power figures are model parameters (6.4 W active, 2.7 W idle, a nominal fog-node envelope) until replaced by a meter reading (Section V-H); we therefore compare energy only between rules under identical parameters.

C. STEP 5: STATISTICAL PROTOCOL

Every configuration is run with independent ns-3 run numbers $1, \ldots, n$ (seed 42), with $n = 20$ for the baseline and n stated for every other experiment, varying sender start offsets and telemetry jitter. Rules compared at the same run number see identical traffic, so comparisons are paired: we use the Wilcoxon signed-rank test and correct with Bonferroni across each family of tests, and report means with standard deviations and 95 % confidence intervals (t distribution, $n-1$ degrees of freedom). Every raw run is stored as one line of key-value output before aggregation, so every number in this paper can be re-derived from the logs.

D. REUSE

Nothing in steps 1 to 5 is specific to HIDS-IoMT. Another detector enters by supplying its decision rule (a function from per-source state and node load to a violation) and its per-flow cost; another dataset enters through steps 2 and 3 unchanged. The simulator, runner and analysis scripts are released (Section VIII).

IV. APPLYING THE FRAMEWORK

A. SYSTEMS AUDITED

HIDS-IoMT [1] is the primary subject. For the broadened audit we screened ten open-access IoMT IDS papers from 2024 to 2026 and retained the two whose confusion matrices are given as numbers rather than images, with metrics for the same experiment: XAI-XGBoost [2] and a leakage-free Nyström sampling study [3], both evaluated on WUSTL-EHMS-2020.

B. DATASETS

IoTID20 [14] (625,783 flow records, CICFlowMeter features), Edge-IIoTset [15] (the 157,800-row file released for machine learning, per-packet tshark fields), WUSTL-EHMS-2020 [13] (16,318 records, Argus flow features plus eight biometric fields) and, for duplicate measurement only, the Friday DDoS day of CICIDS2017 [16]. IoTID20 figures in this paper come from the project's own pipeline run; the other three were measured for this revision from mirrored copies whose row and class counts match the published descriptions.

C. THRESHOLD ARCHITECTURES

All rules decide the same question for each inspected flow: is this source's inter-message gap too short?

- **A, flat** (the base paper): violation if the gap is below $\tau_0 = 500$ ms, for every device.
- **B, congestion-coupled EWMA**: violation if the source's EWMA gap ($\alpha = 0.3$) is below $\tau_0(1 - L)$, so detection loosens as the fog queue fills. This is our reconstruction of a congestion-adaptive rule, not any published code.
- **C, criticality-weighted** (our proposal): violation if the gap is below $\text{clamp}(\tau_0\, w_d\, (1{+}0.3(L{-}0.5)), 50, 2000)$ ms with device-class weights $w_d \in \{0.6, 0.8, 1.0, 1.3\}$ from life-critical to non-clinical.
- **B′, mitigated B** (Section V-F): B with a hysteresis floor, a dual fixed threshold, or a capacity-coupled threshold.

Every rule quarantines a source after $p = 3$ violations unless stated.

D. NETWORK SCENARIOS

The baseline is the Review 2 scenario: 20 legitimate wearables (600 ms telemetry, ±40 % jitter), 60 attacker-bots (5 ms flood), 4 fog nodes, star topology over 10 Mb/s access and 100 Mb/s backhaul links, 30 simulated seconds, per-flow inspection cost 500 µs, balanced device mix. The scenario matrix varies one factor at a time from this baseline: wearables $\{10, 20, 40, 80\}$, attacker-bots $\{20, 40, 60, 80, 120\}$, fog nodes $\{1, 2, 4, 8\}$, topology (star, clustered wards with half the bots in one ward, a 4×4 switch grid, a random geometric switch graph) and device mix (balanced, ICU-heavy 50/30/15/5 %, periphery-heavy 5/15/30/50 %), each at 500 µs and at 2,000 µs per flow. The full factorial would need 960 scenarios per architecture and inspection cost; we run the 16 one-factor scenarios and state the interaction we test separately (inspection cost against attack scale, Section V-F).

E. SIMULATOR VALIDITY

The Review 3 simulator extends the Review 2 program. With every new option at its default it is the same program, and on the same ns-3 [35] build both produce identical detection counts (checked for all three rules at run 1). Moving the unmodified Review 2 program from the ns-3 build used in Review 2 to ns-3.44 changed its single-seed head-to-head counts (Table 2): architecture B went from 8 to 10 quarantined wearables and C from 9 to 10. Nothing in the model changed; the random stream mapping did. This is the strongest argument we have for the multi-seed protocol, and every network-level result below is a 20-seed distribution.

Table 2. The same Review 2 program, run number 1, on two ns-3 builds. Single-seed results are not stable across simulator versions.

ns-3 build	A	B	C
Review 2 build	327 / 60 / 20	303 / 36 / 8	296 / 29 / 9
ns-3.44	321 / 60 / 20	300 / 39 / 10	291 / 30 / 10

Cells: alerts / false-positive alerts / wearables quarantined.

Table 3. Re-derivation from the confusion matrices published in [1].

Quantity	IoTID20	Edge-IIoTset
Printed in [1] (%)		
Accuracy	99.927	99.941
Precision	99.913	99.942
Recall	99.990	99.900
Re-derived, Normal as positive (%)		
Accuracy	99.9265	99.9413
$TP/(TP+FN)$, i.e. recall	99.9126	99.9415
$TP/(TP+FP)$, i.e. precision	99.9898	99.8999
Recomputed, Attack as positive (%)		
Precision	99.735	99.966
Recall	99.969	99.941
F1	99.852	99.954

V. RESULTS

A. AUDIT OF REPORTED METRICS

1) HIDS-IOMT

Section III-G-4 of [1] states that SMOTE balancing yields 2,727,996 Edge-IIoTset instances, and its binary split (2,182,396 training, 272,800 testing, 272,800 validation) sums to exactly that figure; the IoTID20 split sums to 1,170,686 against a stated 1,170,684. The partition is drawn from the oversampled population (check c). Its Section IV-C defines a true positive as a correctly classified *normal* instance, and both of its Equations 7 and 8 are labelled recall, with the second being the precision formula. Table 3 applies them to its published confusion matrices (its Figs. 11c and 12c): accuracy reproduces to four decimals, the printed precision is recovered only as $TP/(TP+FN)$ and the printed recall only as $TP/(TP+FP)$ (checks a and b). With Attack as the positive class, as is standard for a detector, IoTID20 precision is 99.7351 % rather than the printed 99.913 %.

2) AUDIT OF ADDITIONAL SYSTEMS

Table 4 applies the same checks to two more IoMT IDS. XAI-XGBoost [2] reports accuracy 99.22 %, precision 98.35 %, recall 99.91 % and F1 99.12 % on WUSTL-EHMS-2020; all four reproduce exactly from its confusion matrix (TP 2,262, TN 2,836, FP 38, FN 2) with Attack as positive, so checks a, b and e pass. Check c fails on two independent pieces of arithmetic. Its balanced dataset has $14{,}271 + 11{,}417 = 25{,}688$ records and 20 % of that is 5,138, exactly its test support; and that test fold holds 2,264 attack records while the whole raw dataset holds only 2,046. At least 218 test attacks must therefore be synthetic, and the oversampling preceded the split. The Nyström study [3] splits first, oversamples only the training fold, and passes checks a, b, c and e; its leakage-free WUSTL-EHMS-2020 attack recall is 46.91 % and F1 62.34 %. A third screened paper [4] also calls correctly classified normal records true positives, but prints too little to complete the checks. No paper of the three mentions duplicate records (check d); Section V-B measures whether that matters for each dataset.

Table 4. Audit of three published IoMT IDS. C: confirmed, no: not present, n/m: not mentioned.

Check	[1] IoTID20, Edge	[2] WUSTL	[3] WUSTL, ECU
a. Positive-class inversion	C	no	no
b. Transposed precision/recall	C	no	no
c. Partition after oversampling	C	C	no
d. Duplicates handled	n/m	n/m	n/m
e. Irreproducible metric	no	no	no
Reported F1 on its main dataset (%)	99.95†	99.12	62.34

† as printed, under the inverted convention.

Table 5. Exact duplicates and train/test twins along each dataset's feature funnel. Twin: share of test rows with an identical training row under a random 80/20 split (mean of 20).

Feature space	Feat.	Dup. %	Distinct	Twin %
Edge-IIoTset, 157,800 rows				
all raw features	61	0.52	156,986	0.52
minus address and time identifiers	56	3.43	152,381	3.86
minus per-connection nonces and ports	45	50.35	78,353	51.02
minus payload and content columns	38	98.93	1,693	99.47
modelling space, placeholders normalised	30	98.95	1,661	99.48
IoTID20, 625,783 rows (project pipeline)				
all 86 raw columns	86	26.20	461,837	n/a
modelling space (625,415 rows)	69	58.18	261,531	n/a
WUSTL-EHMS-2020, 16,318 rows				
all raw, or identifiers removed	43/35	0.00	16,318	0.00
CICIDS2017, Friday DDoS day only (225,745 rows)				
all raw columns	31	0.00	225,743	0.00
minus identifiers	26	6.26	211,612	7.54

Two observations follow. The metric-definition errors (a, b) did not recur and appear specific to one paper. The ordering error (c) recurred, and it coincides with the size of the reported result: the paper that splits first reports F1 of 62 % on the same dataset where the paper that oversamples first reports 99 %. Section V-B isolates that ordering experimentally on the same data.

B. PREVALENCE OF DATA LEAKAGE

1) DUPLICATES ARE A PROPERTY OF THE FEATURE SPACE

A duplicate rate is not a property of a file. It is a property of the columns a pipeline keeps, and a published figure that does not name its columns cannot be checked. Table 5 measures each dataset along the funnel a careful pipeline follows, from every raw column to the space a model is trained on, and adds the quantity that matters for evaluation: under a random stratified 80/20 split, the share of test rows whose exact feature vector also occurs in training (mean over 20 splits).

On Edge-IIoTset the duplicate rate moves from 0.52 % to 98.95 % as identifier, nonce and payload columns are removed, and each removal is one a careful pipeline should make: ports with a distinct value on a third of all rows are connection identifiers, and payload columns contain the attack strings themselves [23]. Both published figures are correct, the 0.52 % reported over all columns [23] and the 98.95 % our earlier report gave for the modelling space; they measure different populations, and the second is the one a model sees. In that space DDoS_UDP's 14,498 rows and DDoS_ICMP's 14,090

Table 6. Effect of partitioning order, 20 stratified partitions per row (mean ± sd). Attack is positive. Synth.: share of the test fold that is synthetic.

Data	Regime	F1	Precision	Recall	Synth.
WUSTL	LEAKY	0.959 ± 0.003	0.962	0.957	42.7 %
	ORDER = HONEST	0.673 ± 0.014	0.724	0.629	0
Edge	LEAKY	0.941 ± 0.001	0.994	0.894	40.9 %
	ORDER	0.943 ± 0.001	0.998	0.893	0
	HONEST	0.938 ± 0.009	0.961	0.916	0

rows are each a single vector, and 99.5 % of test rows have a training twin. The step from 38 to 30 features also shows the serialisation artefact at work: normalising the two spellings of a missing value merges 32 vectors that had been separated only by spelling, and each of them then carries both labels. IoTID20 rises from 26.2 % in raw rows, consistent with the 164,087 duplicates removed by Alsulami *et al.* [24], to 58.2 % in its modelling space. WUSTL-EHMS-2020 contains no duplicates at all, so for the two papers that use it check (d) is moot and the ordering check is the one that matters.

2) OVERSAMPLING ORDER, ISOLATED OVER 20 PARTITIONS

Table 6 isolates the partitioning order on two datasets with a fixed learner (random forest, 100 trees). LEAKY applies SMOTE [32] to the whole dataset and then splits, the order confirmed in [1] and [2]; ORDER splits first and oversamples only the training fold; HONEST additionally removes duplicates in the modelling space before splitting.

On WUSTL-EHMS-2020 the order alone moves F1 from 0.673 to 0.959 (paired Wilcoxon over 20 partitions, $p = 2 \times 10^{-6}$ raw, 2×10^{-5} after Bonferroni over 12 tests), with 42.7 % of the leaky test fold synthetic. The honest figure sits close to the leakage-free F1 of 62.34 % that [3] report on the same dataset with a different learner, and the leaky figure close to the 99.12 % of [2]: three independent pipelines, one effect. On Edge-IIoTset the order barely matters for F1, because almost every test row already has a training twin, and removing duplicates is what moves the result: precision falls from 0.998 to 0.961 and the false-positive rate on normal traffic rises from 0.8 % to 13.3 % (precision difference, Bonferroni-corrected $p = 2 \times 10^{-5}$). The two datasets fail in different ways, which is why the audit needs both checks.

3) HOW COMMON IS THE LEAKY ORDER?

Of the four pipelines whose order we could establish, three oversample before partitioning ([1], [2], and the prior work in our own laboratory [12]) and one does not [3]. Four is too few to estimate a prevalence, and we do not; the point is that the leaky order is easy to find in recent IoMT work and that, where it is used, its effect can exceed the margin separating published methods by an order of magnitude.

[PENDING: IoTID20 Table 3 over 20 partitions: team_scripts/R3_multiseed_tables.py –dataset iotid20 –table 3. Single-partition values from Review 2: LEAKY F1 0.9986 (46.6 % synthetic), ORDER 0.9994, HONEST 0.9980.]

C. HEAD-TO-HEAD UNDER 20 SEEDS

Table 7 reports all six rules in the baseline scenario at two inspection costs, 20 seeds each. Fig. 2 shows four of the measures with 95 % confidence intervals.

At 500 µs every rule blocks all 60 attackers in all 20 seeds; by detection rate the rules cannot be told apart. They differ in what else they block. The flat rule of [1] quarantines all 20 wearables in every seed, so wearables deliver 21.6 % of their telemetry before being cut off. B quarantines 9.0 ± 2.2 and C 9.85 ± 0.37; the difference between B and C is not significant (paired Wilcoxon $p = 0.08$ raw, 1.0 after Bonferroni). C raises significantly fewer false-positive alerts than B (29.8 ± 0.5 against 36.6 ± 4.5, Bonferroni $p = 0.007$) and does so with about a ninth of B's standard deviation, while B keeps more wearable traffic flowing (delivery ratio 0.709 against 0.647, Bonferroni $p = 0.012$). C is therefore the more predictable rule and the quieter one, not the dominant one, and the Review 2 single-seed reading that B quarantines one wearable fewer than C does not survive multiple seeds as a difference.

At 2,000 µs B fails in all 20 seeds: it blocks 0.05 of 60 attackers on average, the IDS queue overflows by 2.8×10^5 packets, the wearables' mean delay rises from 3.6 ms to 323 ms and their delivery ratio to 0.18, and the fog nodes use 2.2 times the modelled energy of any other rule, inspecting a flood they never stop. A and C are unaffected in detection; their wearable delay rises to 21 and 10.5 ms because wearable packets queue behind flood packets until the bots are quarantined, 110 ms after the attack starts. None of these quantities, delay, delivery, energy or quarantine, exists in a dataset evaluation. Quarantine signalling is negligible for every rule (0.06 to 0.07 % of bytes wherever the rule quarantines anything, and about zero for B when it latches).

D. SCENARIO MATRIX

Fig. 3 varies one factor at a time from the baseline at 500 µs per flow (10 seeds per cell, 20 for the baseline; 35 cells first run under an earlier, larger schedule keep 12 or 20 seeds; full table in the repository).

Three results hold across all 16 scenarios. First, the flat rule of [1] quarantines 100 % of the wearables in every one of them, whatever the ward size, attack scale, fog capacity, topology or device mix: its collateral damage is a property of the rule, not of the scenario. Second, rule C blocks every attacker in every scenario. Third, every rule that blocks attackers at all blocks all of them; no scenario separates the rules by detection rate except where B fails.

At this cost B fails in two scenarios although it was safe in the baseline. With a single fog node the whole flood lands on one queue, B latches off in 10 of 10 seeds and detects nothing (wearable delivery 0.20). On the random topology each leaf is served by its nearest fog node, which concentrates bots on some nodes; B detects 87 % of attackers on average and wearable delivery falls to 0.63. The latch-off is therefore triggered by *placement*, not only by inspection cost, which is a failure mode no dataset and no single-topology simulation would show. Each of the three mitigations restores 100 % detection in both scenarios.

Rule C's collateral damage depends on the device mix, which is what its weights encode. Its quarantine rate is 49 % of wearables in a balanced ward, 20 % in an ICU-heavy ward and 79 % in a periphery-heavy ward. In every mix, no life-critical or high-criticality device was ever quarantined (0 of 5 and 0 of 5 in the balanced ward, 0 of 10 and 0 of 6 in the ICU-heavy ward, 0 of 1 and 0 of 3 in the periphery-heavy ward); every quarantine fell on standard or non-clinical devices. B, hystere-

Table 7. Baseline scenario (20 wearables, 60 bots, 4 fog nodes, star), 20 seeds per row, mean (and sd where it is informative). Bots: of 60 blocked. Quar.: wearables quarantined of 20. PDR: wearable delivery ratio. Delay: wearable mean, including inspection queueing. Energy: modelled fog inspection energy over 30 s. Latched: runs with <50 % of bots blocked and queue overflow.

Rule	Bots	Quar.	FP alerts	PDR	Delay (ms)	Energy (J)	Latched
500 µs per flow							
A flat	60.0	20.00 ± 0.00	60.0 ± 0.0	0.216	3.6	325	0/20
B EWMA	60.0	9.00 ± 2.15	36.6 ± 4.5	0.709	3.6	326	0/20
C criticality	60.0	9.85 ± 0.37	29.8 ± 0.5	0.647	3.6	326	0/20
B′ hysteresis	60.0	9.00 ± 2.15	36.6 ± 4.5	0.709	3.6	326	0/20
B′ dual	60.0	9.00 ± 2.15	36.6 ± 4.5	0.709	3.6	326	0/20
B′ capacity	60.0	8.65 ± 2.11	35.5 ± 4.5	0.738	3.6	326	0/20
2,000 µs per flow							
A flat	60.0	20.00 ± 0.00	60.0 ± 0.0	0.216	21.4	333	0/20
B EWMA	0.1	0.00 ± 0.00	6.5 ± 2.0	0.182	322.8	739	20/20
C criticality	60.0	9.85 ± 0.37	29.8 ± 0.5	0.645	10.5	336	0/20
B′ hysteresis	60.0	8.70 ± 2.25	35.8 ± 4.6	0.729	9.9	337	0/20
B′ dual	60.0	8.70 ± 2.25	35.8 ± 4.6	0.729	9.9	337	0/20
B′ capacity	60.0	8.90 ± 2.20	35.9 ± 4.7	0.716	10.0	337	0/20

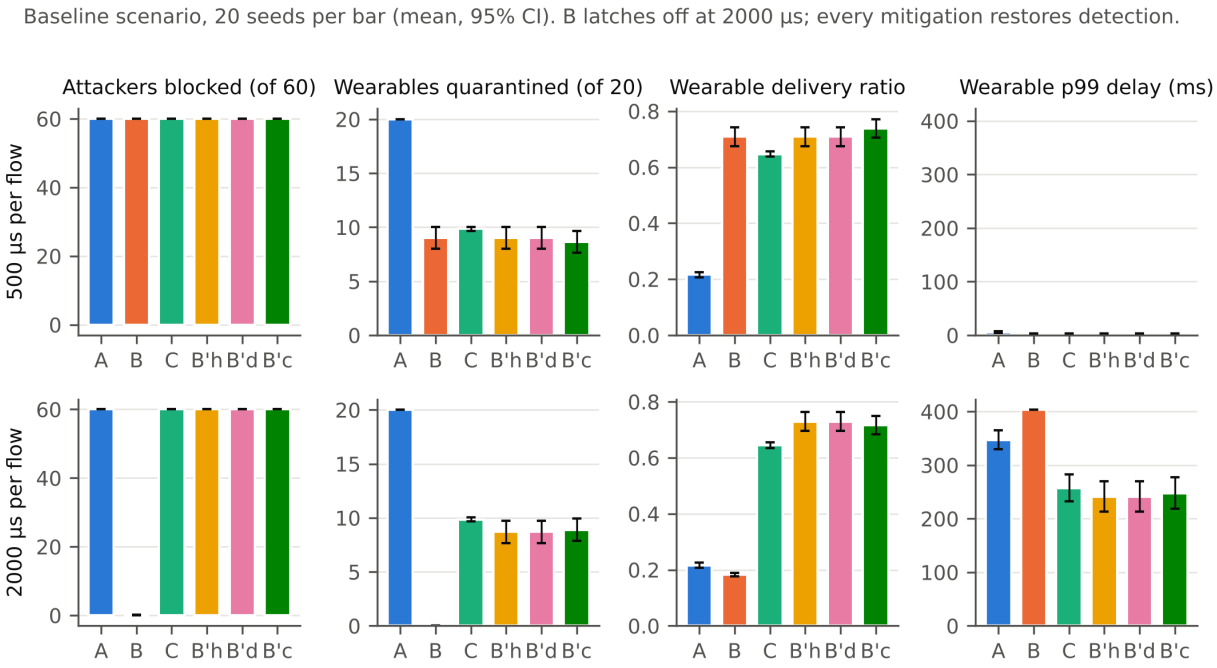


Figure 2. Baseline scenario, 20 seeds per bar (mean and 95 % CI). Top: 500 µs per flow. Bottom: 2,000 µs. B'h, B'd, B'c: B with hysteresis, dual and capacity-coupled mitigation.

sis and dual threshold quarantine 45 to 47 % in all three mixes, because they ignore device role. The capacity-coupled variant quarantines between 0 % (20 bots) and 47 % (80 bots): its utilisation estimate counts all arriving traffic, including floods from bots it has already quarantined, so below its knee it relaxes in proportion to attack pressure, and its collateral damage varies with the attack rather than with the ward.

E. SENSITIVITY ANALYSIS

Fig. 4 varies one parameter at a time around the baseline (10 seeds per level): the base threshold $\tau_0 \in \{100, 250, 500, 1000\}$ ms, B's smoothing $\alpha \in \{0.1, 0.3, 0.5, 0.7\}$, C's weight scheme (frozen $\{0.6, 0.8, 1.0, 1.3\}$, uniform, linear $\{0.7, 0.9, 1.1, 1.3\}$, exponential $\{0.25, 0.5, 1, 2\}$) and the inspection cost $\{57.9, 500, 1000, 2000, 14{,}542.7\}$ µs.

The base threshold matters most, for every rule. At $\tau_0 = 100$ or 250 ms all three rules quarantine no wearable and still block all 60 bots, because the wearables' shortest legitimate gap is 360 ms and the bots send every 5 ms; at 1,000 ms all three quarantine all 20. The base paper's collateral damage is thus a consequence of one global constant sitting above some devices' telemetry gaps. That conclusion holds against a fast flood only; Section V-G repeats the sweep against attackers who send at wearable-like rates, where lowering τ_0 stops being free.

For B, α is the second lever: from 0.1 to 0.7 its quarantine count rises from 5.4 to 19.9, because a less smoothed gap estimate reacts to single short gaps in jittered telemetry. No value of α prevents the latch-off: at 2,000 µs B latched in 10 of 10 seeds for every α tested. Inspection cost decides B's detection outright (all or nothing) and leaves A and C untouched up to 14.5 ms per flow.

For C, the weight scheme is the second lever. Uniform weights make C behave like a flat rule (19.3 wearables quarantined, 59 false alerts); the linear scheme quarantines 12.4; the

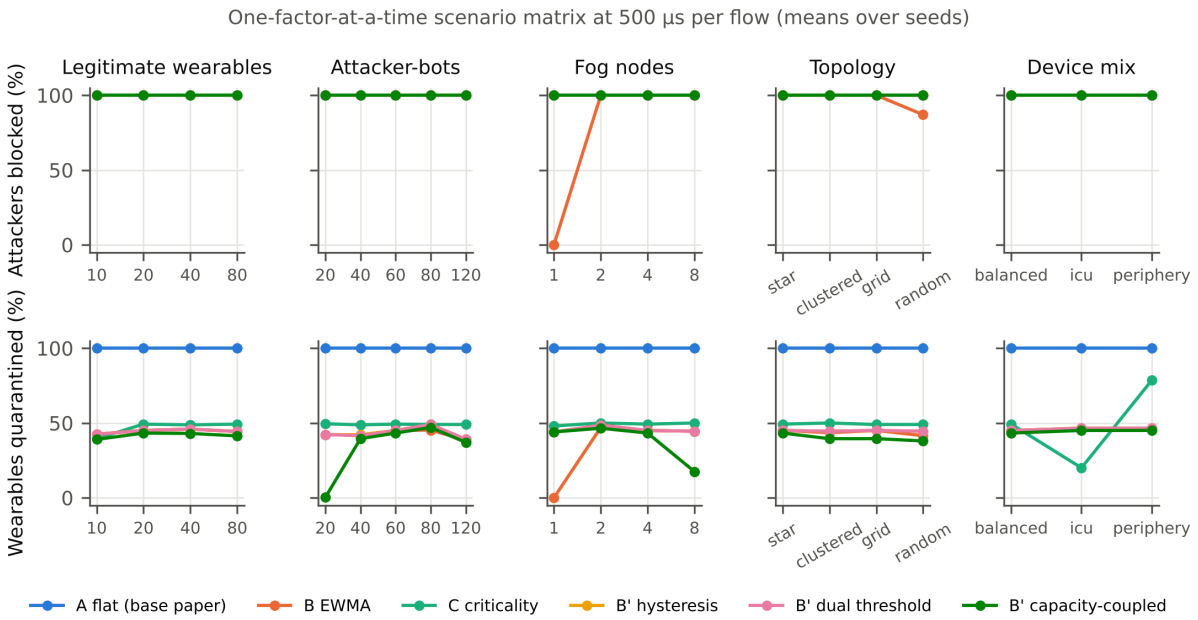


Figure 3. Scenario matrix at 500 µs per flow, means over seeds. Top: attackers blocked. Bottom: wearables quarantined. Lines that coincide are hidden behind the last one drawn; in the top row every rule except B is at 100 % in every cell.

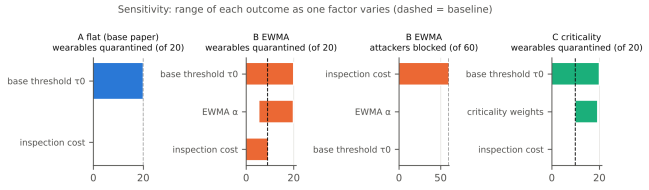


Figure 4. Sensitivity. Each bar spans the outcome's range as one factor varies; dashed line: baseline.

frozen and exponential schemes both quarantine 9.8, because under both every life-critical and high-criticality device stays below the gap threshold and every non-clinical one above it. What the weights must get right is which side of the telemetry distribution each class falls on, not their exact values.

F. LATCH-OFF IN CONGESTION-COUPLED THRESHOLDS

1) MECHANISM

Rule B's threshold is $\tau_0(1 - L)$. A flood fills the fog queue, L approaches 1, the threshold approaches zero, and no inter-arrival gap can be shorter than zero. The loop can run either way. If enough bots are quarantined early, their load disappears, the queue stays short, the threshold stays high and the rest are caught. If they are not, the queue saturates, the threshold collapses, nothing can violate it and the load never falls. The detector does not degrade under load; it switches off and stays off. Review 2 showed this as a single-seed discontinuity in patience: 60/60 bots blocked with $p \leq 2$, 0/60 with $p \geq 3$.

2) THE LATCH-OFF BOUNDARY

To locate the boundary we swept inspection cost $\{500, 1000, 2000, 3000, 5000\}$ µs, attacker-bots $\{20, 40, 60, 80, 120\}$, patience $p \in \{1, \ldots, 5\}$ and smoothing $\alpha \in \{0.1, 0.3, 0.7\}$: 375 runs of 20 simulated seconds, one seed per cell to fit the compute budget (Fig. 5).

The outcome is all or nothing. Of the 375 runs, 208 blocked every bot, 152 blocked none and only 15 ended in between: there is no regime in which B degrades gracefully. At the Review 2 patience of 3 the detector works at 500 µs for every attack scale up to 120 bots, latches at 1,000 µs only with 120 bots, from 60 bots at 2,000 and 3,000 µs, and from 40 bots at 5,000 µs; 20 bots never latch it at any cost we tried. At 2,000 µs and 60 bots, $p \leq 2$ works and $p \geq 3$ latches, which reproduces the Review 2 discontinuity exactly.

Patience moves the boundary and smoothing barely does. The share of cells that latch grows from 24 % at $p = 1$ to 40, 44, 51 and 52 % at $p = 2$ to 5, because each extra violation required before quarantine gives the queue more time to fill. The three values of α agree in 72 of the 75 (patience, cost, attack scale) cells; the three disagreements are at $p \geq 4$, where the slowest smoothing latches and the fastest does not.

Offered utilisation alone does not predict the outcome. With ρ the per-fog arrival rate times the inspection cost, every cell with $\rho \leq 2$ kept working and every cell with $\rho \geq 12$ latched; between those the outcome depends on patience. A detector can therefore survive an offered load ten times its capacity if it quarantines fast enough, and fail at three times capacity if it waits (ρ values are computed from the configured rates, before any quarantine). The latch-off is a race between quarantine and queue growth, which is why the mitigations act on the threshold's floor or on its coupling rather than on capacity alone.

3) MITIGATIONS

We implemented all three options and evaluate each as a variant B′ of rule B, changing nothing else.

- **Hysteresis floor** (option B). The threshold may not fall below $0.3\tau_0$, and once load exceeds 0.8 it is held at that floor until load falls below 0.2, so it can neither collapse nor oscillate at the boundary.
- **Dual threshold** (option C). A source violates if its EWMA gap is below the adaptive threshold *or* its raw gap is below

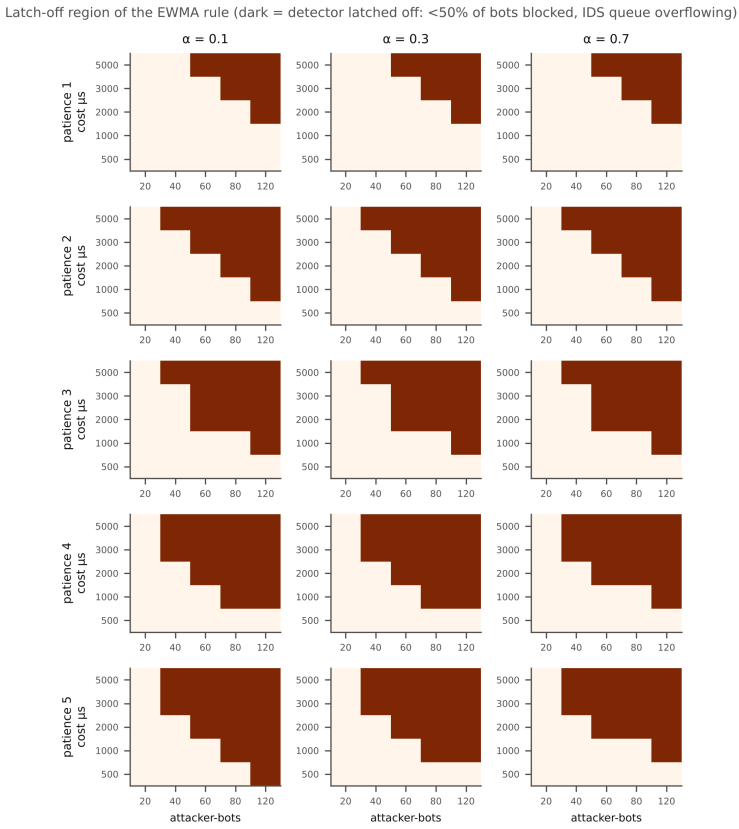


Figure 5. Latch-off region of rule B. Rows: patience; columns: smoothing α; within each panel, inspection cost against attack scale. Dark cells latched off (fewer than half of the bots blocked, IDS queue overflowing). One seed per cell.

a fixed 100 ms, so a flood is caught whatever the load.

- **Capacity coupling** (option A). The rule estimates offered utilisation ρ (arrivals in the last second times the per-flow cost) and relaxes as $\tau_0(1-\rho)$ only while $\rho < 0.8$. At or above that boundary, demand beyond capacity is itself evidence of a flood, and the threshold returns to the strict τ_0.

In the baseline scenario at 2,000 µs, where B latches in 20 of 20 seeds, each mitigation blocks 60 of 60 bots in 20 of 20 seeds (Table 7; raw $p = 1.2 \times 10^{-5}$ for each against B, 4×10^{-4} after Bonferroni over the 36 pairwise tests among B and its variants), restores the wearable delivery ratio from 0.18 to between 0.72 and 0.73 and the mean wearable delay from 323 ms to about 10 ms. At 500 µs, where B does not latch, hysteresis and dual threshold produce results identical to B in every seed: they cost nothing when the network is healthy. Capacity coupling quarantines slightly fewer wearables than B at 500 µs (8.65 against 9.0, not significant after correction) because it relaxes on offered rate rather than on queue length.

4) VALIDATION ACROSS THE SCENARIO MATRIX

We repeated the full 16-scenario matrix at 2,000 µs per flow, the cost at which B latches in the baseline (5 seeds per cell, 20 for the baseline; Fig. 6). With five seeds a paired Wilcoxon test cannot fall below $p = 0.0625$, so we report counts of runs rather than p-values.

At this cost B is the exception rather than the rule. It blocks every attacker in only three of the 15 non-baseline scenarios (20 and 40 bots, 8 fog nodes), each of which lowers the load per fog queue. It latches off, blocking fewer than half of the attackers, in all 5 seeds of 11 scenarios: every ward size, 80 and 120 bots, 1 and 2 fog nodes, the grid and random topologies and both skewed device mixes. On the clustered topology it blocks exactly 30 of 60 bots in every seed, consistent with the fog nodes that carry the clustered traffic latching while the others keep working. Across the matrix, 55 of B's 75 runs latch.

Each mitigation blocks every attacker in every one of its 75 runs, as do A and C. Averaged over the scenarios in which B latches, the mitigated rules raise wearable delivery from 0.19 to 0.76 and lower mean wearable delay from 265 ms to about 10 ms and modelled fog energy from 623 J to 301 J. Their collateral quarantine ranges from 26 % (one fog node) to 50 % (80 bots) of wearables, against 100 % for A in every scenario and 20 % to 79 % for C depending on the device mix. The three mitigations differ from one another by at most four percentage points of quarantine in any scenario, so the choice between them can rest on implementation cost: the hysteresis floor and the dual threshold need no capacity estimate and leave B unchanged when the network is healthy.

G. THE LIMITS OF THRESHOLD ADAPTATION

1) WHY THE CRITICALITY RULE MISSED ITS TARGET

Review 2 measured rule C in the detection pipeline against a 30 % false-positive-reduction target and found at best 10.8 %: an ICU-heavy segment traded 23.4 % more false positives for 48.6 % fewer missed attacks, a periphery-heavy segment the reverse, and the total error count (FP+FN between 294 and 311) barely moved. We now test the explanation offered then, that a threshold rule moves the operating point along a fixed trade-off curve and cannot move the curve.

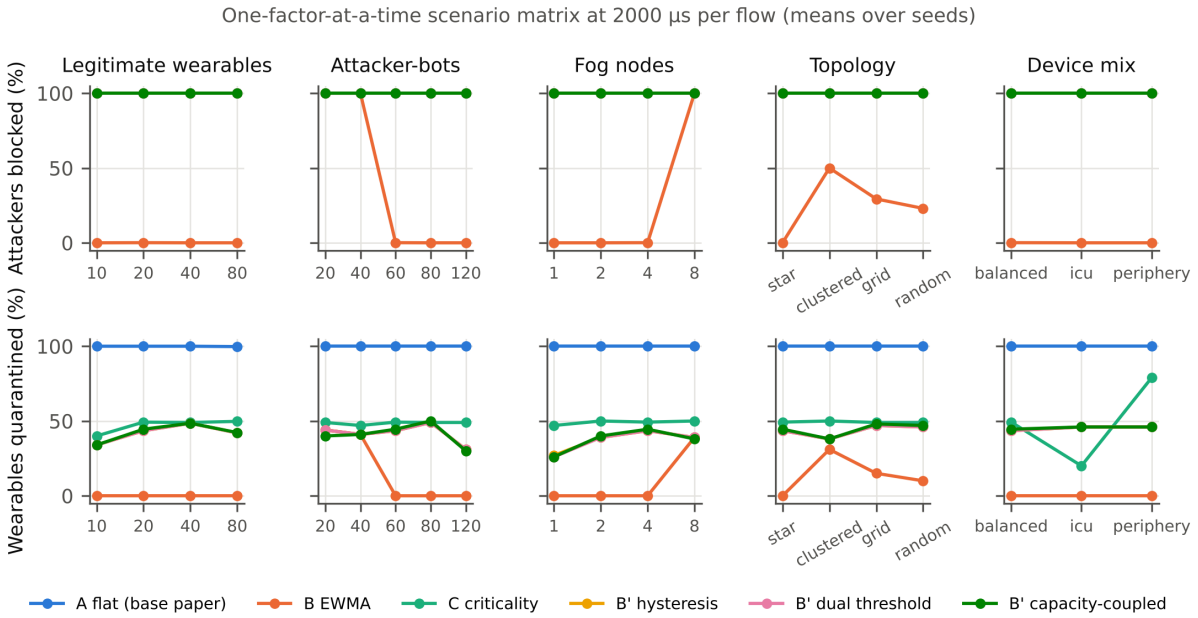


Figure 6. Scenario matrix at 2,000 µs per flow, means over seeds. Top: attackers blocked. Bottom: wearables quarantined. B is the only rule that falls below 100 % detection; the three mitigated rules coincide in most cells.

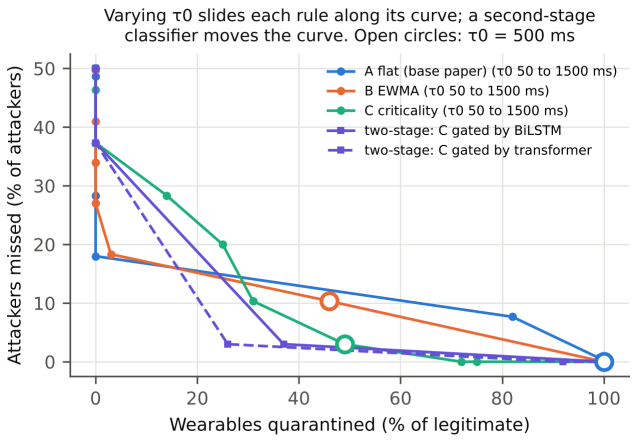


Figure 7. False quarantine against missed attackers as τ_0 varies from 50 to 1,500 ms, with half the bots sending at wearable-like rates. Open circles: $\tau_0 = 500$ ms. Purple: rule C gated by a measured classifier branch (independent errors).

2) THE TRADE-OFF CURVE

The baseline flood is trivially separable from telemetry (Section V-E), so the curve needs attackers who resemble the devices. Half of the 60 bots now send at a fixed period drawn uniformly from 300 to 700 ms with ±40 % jitter, overlapping the wearables' 360 to 840 ms, and the other half flood. We sweep τ_0 from 50 to 1,500 ms for each rule (5 seeds per point) and plot wearables quarantined against attackers missed (Fig. 7). Every threshold rule spans the same two corners: no quarantine with half the attackers missed (the slow half), and every wearable quarantined with none missed. None comes near the origin. Varying τ_0 slides each rule along its own curve, and the flat rule's curve has almost no middle: it jumps from 0 % quarantined and 18 % missed at 350 ms to 82 % quarantined and 7.7 % missed at 400 ms, because one constant cannot separate two overlapping gap distributions. C's curve lies below A's and B's in the middle region, at 31 % quarantined for 10.3 % missed and 49 % for 3.0 %, since device class is information the other rules lack; but C too reaches zero missed attacks only by quarantining 72 % of wearables. Changing the rule's parameter moves the operating point; changing the information the rule has moves the curve, and how far is bounded by that information.

Table 8. Two-stage rule in the baseline scenario, 500 µs, 20 seeds. Reductions in %; positive is fewer.

	Bots /60	**Quar.** /20	**FP** alerts	**ΔFP** vs C	**ΔQuar.** vs C	**ΔFP** vs A
C alone	60.00	9.85	29.8	+0.0	+0.0	50.3
C + BiLSTM (indep.)	60.00	7.90	26.8	+10.2	+19.8	55.4
C + BiLSTM (per device)	59.50	5.30	16.0	+46.3	+46.2	73.3
C + transformer (indep.)	60.00	4.75	19.9	+33.4	+51.8	66.9
C + transformer (per device)	58.30	2.15	6.5	+78.4	+78.2	89.3
C + ensemble (indep.)	60.00	6.55	24.3	+18.5	+33.5	59.5
C + ensemble (per device)	58.75	3.45	10.5	+64.9	+65.0	82.6

3) A TWO-STAGE RULE

Stage 1 is rule C. Stage 2 is a flow classifier: a violation counts toward quarantine only if the classifier also flags the flow. Because no neural network runs inside ns-3, stage 2 is represented by the measured operating point of a trained branch on IoTID20 test windows (Table 13 pipeline, Attack positive): BiLSTM recall 0.9916 at a false-positive rate of 0.5103, transformer 0.9735 at 0.2194, full ensemble 0.9831 at 0.3529. Classifier errors are drawn either independently per flow, or once per device, which bounds the case where a classifier's mistakes stick to a device.

Fig. 7 shows the gated rule on the mimicry scenario. At the same 3.0 % of attackers missed, C alone quarantines 49 % of wearables, C gated by the BiLSTM 37 % (24.5 % fewer) and C gated by the transformer 26 % (47 % fewer): the gate moves the curve toward the origin, and only the transformer gate moves it by more than 30 % at constant missed attacks.

Table 9. Measured per-window inference cost, x86-64 2-vCPU host, LiteRT float32, single thread (ms). Load: background CPU-bound processes.

	CNN	BiLSTM	Transformer	Ensemble
p50, idle	0.026	0.330	0.057	0.457
p95, idle	0.045	0.444	0.105	0.558
p50, both cores loaded	0.029	0.324	0.058	0.540
p95, both cores loaded	0.047	0.503	0.093	4.493
p99, both cores loaded	0.093	4.457	0.230	4.660

Table 8 gives the same rules in the baseline scenario over 20 seeds. With independent errors, gating C with the transformer cuts false-positive alerts by 33.4 % and quarantined wearables by 51.8 % relative to C alone while still blocking all 60 bots in every seed, meeting the 30 % target relative to C and reaching 66.9 % relative to the flat rule. Gating with the BiLSTM, the branch the roadmap proposed, reaches only 10.2 % and 19.8 %, because that branch flags 51 % of normal windows. When errors stick to devices the reductions grow (up to 78 %) but between 0.5 and 1.7 of the 60 bots escape in an average seed, because a bot the classifier misjudges once is misjudged on every packet. Within a rule, the threshold only chooses a point on the curve; a second, independent source of evidence is what moves it, and how much it helps depends on the false-positive rate of that evidence on normal traffic, not on its accuracy, which is above 0.92 for all three branches.

4) REVISED CONCLUSION ON RULE C

We therefore restate the Review 2 result in two parts. In the detection pipeline, the criticality weights act as per-class thresholds on a single classifier score; they reallocated errors between false positives and missed attacks and did not reduce false positives by 30 % at constant missed attacks, and no threshold on that one score can, because it only selects a point on that score's curve. In the network, device class is information the flat rule does not have, and there it changes the curve: at 3 % missed attackers C quarantines about half the wearables where the flat rule offers no operating point at all between 0 and 82 %, and at its default settings it never quarantined a life-critical or high-criticality device in any scenario, cost or two-stage variant; raising τ_0 to 1,000 ms, flattening the weights, or the mimicry attack at $\tau_0 \geq 600$ ms does reach those classes. Threshold adaptation has a ceiling set by the information the threshold acts on. Lowering it further takes a second discriminator, and the two-stage result shows that what matters is that discriminator's false-positive rate on normal traffic.

H. IMPACT OF HARDWARE LATENCY ON DETECTION PERFORMANCE

1) MEASURED INFERENCE COST

Inference cost does not depend on weight values, so the frozen three-branch architecture was exported to TensorFlow Lite (float32, BiLSTM unrolled so that only built-in operators are used; 1.1 MB on disk, 305,798 parameters) and timed with the LiteRT interpreter on one thread, 1,000 windows after 50 warm-up windows, at three background CPU loads. The host was a two-vCPU Intel Xeon at 2.1 GHz; it is a reference measurement, not a Raspberry Pi, and is labelled as such everywhere it is used.

The BiLSTM is 72 % of the ensemble's median cost. The median barely moves under contention (0.457 to 0.540 ms) but the 95th percentile grows eightfold, to 4.49 ms, when the inference thread competes for the core. Sustained throughput was 1,918 windows per second at 99 % of one core, and the three models add 14 MB of resident memory. Energy could not be measured on this host, which exposes no power counters; it is reported only from the model in Section III until a metered run is available.

Table 10. Latency and load sweep with measured per-flow costs (x86 host). Attackers blocked (%), and wearable p99 delay (ms) in parentheses; 10 to 20 seeds per cell.

Cost	Rule	20 bots	40	60	80
456.7 µs	A	100 (3.6)	100 (3.7)	100 (4.8)	100 (8.4)
idle p50	B	100 (3.5)	100 (3.5)	100 (3.5)	100 (3.6)
	C	100 (3.5)	100 (3.5)	100 (3.5)	100 (3.6)
4,492.7 µs	A	100 (232)	100 (841)	100 (850)	100 (855)
loaded p95	B	100 (125)	**0** (901)	**0** (901)	**0** (901)
	B′ hyst.	100 (125)	100 (710)	100 (696)	100 (691)
	C	100 (140)	100 (743)	100 (742)	100 (726)

2) SIMULATION WITH MEASURED COSTS

We re-ran the latency and attack-scale sweep of Review 2 with the two measured costs in place of swept ones: the idle median (456.7 µs) and the loaded 95th percentile (4,492.7 µs), at 20 to 80 bots (Table 10). Each run carries its provenance string, naming the host and stating that it is not a Raspberry Pi.

Two conclusions follow. The latch-off is robust to realistic hardware: at the cost measured under contention, B latches in 10 of 10 seeds from 40 bots upward, between the swept 2,000 µs case (latched from 60 bots) and the swept 14,542.7 µs case (from 20), and the hysteresis floor restores full detection at every scale. The blind spot is therefore not an artefact of the swept values. Second, a detector that keeps detecting can still fail the clinical requirement: with the loaded cost, wearable p99 delay rises from 3.5 to 8.4 ms at the idle cost to between 690 and 855 ms at 40 bots or more under A, C and the mitigated rule, because telemetry queues behind the flood at the fog node until the bots are quarantined. The median cost of inference barely changes under contention; its tail decides both the latch-off and the delay, which is why the measurement must report percentiles under load and not a single benchmark figure.

3) RASPBERRY PI MEASUREMENT

The same harness then ran on a Raspberry Pi 4 Model B (Rev. 1.5, four Cortex-A72 cores, 4 GB, Raspberry Pi OS 64-bit, LiteRT 2.3.0, Python 3.13) with the *trained* ensemble rather than the random-weight copy: the float32 export of the project's CNN, unrolled BiLSTM and transformer, 1.42 MB on disk. Before any timing, the device ran 256 real test windows through the three branches and the fusion rule and reproduced the evaluated predictions exactly (100 % argmax agreement, largest probability difference 6.5×10^{-8}). The board reported no under-voltage or thermal throttling before or after the run (`throttled=0x0`; 38.9 to 44.3 °C). Table 11 sets it beside the same trained files timed on the x86 host.

The Pi is 6.4 times slower than the x86 host at the median and sustains 334 windows per second at one full core. The BiLSTM is 69 % of its median cost, as on x86, so the recurrent branch remains the price of the ensemble on an edge device.

Table 11. Measured per-window inference cost of the trained ensemble, LiteRT float32, single inference thread (ms). Loaded: every core busy with a background CPU-bound process (2 on x86, 4 on the Pi).

	x86 host	Raspberry Pi 4
Ensemble p50, idle	0.467	3.003
Ensemble p95, idle	0.550	3.026
Ensemble p50, loaded	0.592	3.014
Ensemble p95, loaded	4.672	6.180
Ensemble p99, loaded	4.732	7.037
BiLSTM p50, idle	0.299	2.079
Sustained windows per second	1,999	334
Resident memory added by the models (MB)	21.8	22.4

Table 12. Latency and load sweep with per-flow costs measured on the Raspberry Pi 4 (trained ensemble). Attackers blocked (%), and wearable p99 delay (ms) in parentheses; 10 seeds per cell.

Cost	Rule	20 bots	40	60	80
3,003.2 µs	A	100 (67)	100 (479)	100 (562)	100 (538)
idle p50	B	100 (11)	100 (341)	**0** (604)	**0** (604)
	B′ hyst.	100 (11)	100 (341)	100 (422)	100 (382)
	C	100 (12)	100 (356)	100 (446)	100 (424)
6,180.1 µs	A	100 (533)	100 (1186)	100 (1187)	100 (1167)
loaded p95	B	100 (408)	**0** (1239)	**0** (1239)	**0** (1239)
	B′ hyst.	100 (408)	100 (1028)	100 (1034)	100 (1034)
	C	100 (431)	100 (1083)	100 (1067)	100 (1076)

The Pi's tail behaves like the x86 host's in kind but not in degree: with up to three of its four cores busy the 95th percentile stays within 0.15 ms of the median, and only when every core is busy does it double, to 6.18 ms. Energy was not measured: the Pi 4 has no power counters and no inline meter was available, so the energy figures in this paper remain modelled.

4) SIMULATION WITH THE RASPBERRY PI'S MEASURED COSTS

We then repeated the latency and attack-scale sweep with the two costs measured on the Pi: its idle median (3,003.2 µs) and its 95th percentile with every core busy (6,180.1 µs), 10 seeds per cell, every run stamped with the board's name (Table 12). On the Pi the congestion-coupled rule fails without any contention at all. At the board's idle median it latches in 10 of 10 seeds from 60 bots upward, which includes the Review 2 baseline of 60 bots: a configuration that worked at the x86 host's idle cost switches itself off on the edge device a fog node would use. Under contention it latches from 40 bots. The hysteresis floor, the flat rule and the criticality rule block every attacker in every cell. Delay is where the hardware shows most. At 40 to 80 bots, wearable p99 delay is 341 to 562 ms at the idle cost and 1.03 to 1.19 s at the loaded cost under the rules that keep detecting, against 3.5 to 8.4 ms on the x86 host at its idle cost; a fog node built on this board would deliver telemetry late during every flood it stops. The effect of hardware latency on detection performance is therefore not a gradual loss of accuracy: below the latch-off boundary detection is unchanged and the cost appears as delay, and above it a congestion-coupled detector stops detecting altogether.

I. DETECTION PIPELINE: PARTITIONS, ENSEMBLES AND GENERALISATION

1) ENSEMBLE ABLATION

The three-branch ensemble (a three-layer 1-D CNN; a two-layer BiLSTM with 64 units per direction; a two-layer, four-head transformer encoder; confidence-weighted fusion) operates on ten-record flow windows built within sessions, with sessions rather than windows assigned to folds. On a single session-level partition (Table 13) the fused ensemble is beaten by its best branch on both datasets, and the best branch differs between them. The three branches agree on 90.9 % of IoTID20 test windows, so fusion acts on 9.1 % of them, and an oracle that resolved every disagreement correctly would gain only 0.026 accuracy over the best branch. **[PENDING: 20-partition version of this table: team_scripts/R3_multiseed_tables.py –table 7 for both datasets, adding mean ± sd and a paired test of ensemble against best branch.]**

Table 13. Window-level accuracy, single session-level partition (Review 2). Attack is positive.

Configuration	IoTID20	Edge-IIoTset
CNN alone	0.9230	**0.9798**
BiLSTM alone	0.9271	0.9201
Transformer alone	**0.9487**	0.8945
Full three-branch ensemble	0.9399	0.9214

2) ATTACKS WITHHELD FROM TRAINING

A detector deployed in a hospital meets attack types it was not trained on. On the honest (deduplicated) data we hold out one attack type at a time, train on normal traffic and the remaining types, and measure recall on the withheld type over five seeds. On WUSTL-EHMS-2020 the detector recognises 1.1 % of withheld data-alteration attacks and 0.09 % of withheld spoofing attacks. On Edge-IIoTset recall on a withheld type is mostly near one extreme: near 100 % for types whose behaviour resembles another training type (DDoS_HTTP, SQL injection, XSS, password, uploading, vulnerability scanning) and near zero for those that do not (backdoor 1.1 %, ransomware 0.7 %), with two in between (MITM 76 %, fingerprinting 80 %), while false alarms on held-out normal traffic stay between 3 % and 23 %. Three Edge-IIoTset types (DDoS_UDP, DDoS_ICMP, port scanning) cannot be held out at all, because after deduplication their few distinct vectors coincide with vectors of other classes. A 99 % in-distribution score says little about the attack a hospital has not yet seen.

3) CROSS-DATASET VALIDATION

The requested IoTID20↔Edge-IIoTset transfer cannot be run as specified. IoTID20 is a table of CICFlowMeter flow statistics; Edge-IIoTset is a table of per-packet tshark protocol fields. Matching column names after normalising spelling, the two share no feature, and neither shares one with WUSTL-EHMS-2020's Argus flow features; only datasets produced by the same extractor overlap (IoTID20 and CICIDS2017 share 20). A model trained on one of these files cannot be evaluated on another without re-extracting both from raw captures with a common tool, which is the experiment we propose rather than one we can claim. We report the held-out-attack test above as the generalisation evidence available within each dataset.

4) INCREMENTAL ADAPTATION

With a replay buffer over 8,646 trainable parameters, recall on a newly introduced attack type rose from 0.9746 to 0.9880

while recall on previously learned types rose from 0.9821 to 0.9863 (forgetting $+0.0042$). Fine-tuning the fusion layer alone, the rejected alternative, raised the false-positive rate from 0.5231 to 0.6280.

J. EXPLANATION LAYER

1) STABILITY AND CERTIFICATION

Each alert carries a SHAP attribution [33], rendered in plain language and split into evidence for and against the verdict. Under a 1 % Gaussian perturbation, SHAP's top-5 features have a Jaccard stability of 0.83 (Spearman $+0.92$; Review 2 quoted 0.87 from an earlier run of the same evaluation, and we report the value the current evaluation script produces), and LIME's [34] 0.27 (Spearman $+0.21$), while the model's own prediction flips in 0 % of cases, so the instability belongs to the explainer. Raising LIME's sample count from 1,000 to 12,000 moves its Jaccard only within 0.16 to 0.38 (0.16, 0.38 and 0.33, and its apparent fidelity falls from $+0.349$ to $+0.023$ as sampling converges. LIME was demoted to an internal cross-check. Each SHAP explanation is certified before display by ablating its top-cited features and checking that the decision moves more than for random features; 82 % of 40 explanations certify, and the rest are shown labelled unverified rather than hidden.

2) HUMAN EVALUATION OF EXPLANATION STABILITY

Certification only helps if a reader notices it. We designed a within-participant reader study with pre-registered hypotheses: (H1) readers state the reason for a certified alert correctly more often than for an uncertified one; (H2) readers trust uncertified alerts less; (H3) contradictory alerts, whose top-ranked evidence argues against the verdict, lead readers to cite counter-evidence as the reason. Ten to fifteen participants without exposure to the model each read 20 real alerts produced by the trained ensemble (8 certified, 6 uncertified, 6 contradictory) in a seeded per-participant order, and for each state the verdict, the system's confidence band, the reason (a checklist of the cited measurements plus two distractors that the alert never cites, and one free sentence), the action they would take, their own confidence and their trust. Time to decision is recorded. A reason is scored correct when the reader selects at least one measurement the alert cites for its verdict and no counter-evidence or distractor, the pass rule fixed before the study. Analysis compares participant-level means per category with paired Wilcoxon tests, Bonferroni-corrected, and actions by category with a chi-square test; free-text reports of unreliable alerts are coded independently by two team members with Cohen's kappa.

[PENDING: Run the protocol (reader_study/PROTOCOL.md; study app; 25 minutes per participant), export the CSV, run reader_study/analyse_reader_study.py and insert participant demographics, accuracy, confidence, time and trust per category here. No result is reported until the sessions have been run.]

VI. DISCUSSION

What the network adds. Every result in Sections V-C to V-H concerns a quantity that does not exist in a dataset evaluation: which legitimate devices are disconnected, whether the detector keeps working when its own queue fills, how long telemetry waits behind a flood, and how much inspection energy an inert detector wastes. Three rules with identical detection rates differ between quarantining none of a ward's life-critical devices and quarantining every device on the ward. A rule that looks acceptable in the baseline switches itself off when the same traffic is routed to fewer fog nodes. A detector that keeps detecting under a realistic tail inference cost still delays wearable telemetry by most of a second. None of this is a criticism of the audited authors' care; it is what the evaluation instrument cannot see.

What the audit adds. The arithmetic checks need no code and no data, and they caught errors in two of three papers. The most consequential, partitioning after oversampling, coincides with the largest reported scores: on WUSTL-EHMS-2020 our own 20-partition replication moves F1 by 0.29 with the order alone, and three independent pipelines land on either side of that gap according to their order. A reader comparing published IoMT detectors on that dataset is largely comparing evaluation protocols.

Duplicates need a feature space. A duplicate rate stated without its columns cannot be checked. Our own earlier Edge-IIoTset figure was challenged for exactly this reason, and both it and the conflicting published figure turn out to be correct for their populations. We recommend that papers report duplicate and train/test-twin rates in the feature space the model is trained on, which is the only one that bears on evaluation.

Design guidance for in-path detectors. Three follow from the results. Do not couple a detection threshold to observed congestion without a floor: a detector in the data path controls the congestion it observes, and the loop can lock in the wrong state. If a rule must adapt to load, couple it to measured capacity, so that overload, which is itself evidence of an attack, tightens rather than loosens it. And state per-device consequences: a rule's quarantine profile by device class is a clinical decision that the criticality weights make explicit. At its default settings C never disconnected a life-critical device in any scenario, but a higher base threshold, flatter weights or a mimicry attack did reach that class, so the guarantee belongs to a configuration, not to the rule.

Negative results. Our ensemble did not beat its best branch, LIME was unstable, the detector did not recognise withheld attack types, a threshold rule alone did not reach its false-positive target in the detection pipeline, and cross-dataset transfer between the requested datasets is impossible without re-extraction. We report these because the literature's ceiling effect is partly made of results like them left unreported.

VII. THREATS TO VALIDITY

Simulation abstraction. The fog IDS is a single-server queue applying a timing rule; no neural network runs inside ns-3. Detector behaviour enters the network model through two measured quantities, per-flow inference cost (Section V-H) and, for the two-stage rule, the branch's measured true- and false-positive rates (Section V-G). The two-stage results assume classifier errors are either independent per flow or fixed per device; real errors lie between, and we report both bounds rather than one.

Scenario coverage. The scenario matrix varies one factor at a time around one baseline, so interactions between factors are not covered except for inspection cost against attack scale, patience and smoothing, which the latch-off phase diagram

sweeps jointly. Topologies are wired switch fabrics; wireless contention, mobility and energy-constrained wearables are out of scope. Energy is modelled from nominal fog-node power figures and is used only to compare rules under identical assumptions.

Seeds and compute. The baseline uses 20 seeds per configuration. To fit the compute budget, the scenario matrix uses 10 seeds per cell at 500 µs and 5 at 2,000 µs, the sensitivity sweep 10, and the latch-off phase diagram one seed per cell over 375 cells with 20-second runs; latching is decided in the first seconds of a run and is close to binary, and its boundary is confirmed by the 20-seed baseline. Every table states its n.

Hardware. Inference cost was measured on a Raspberry Pi 4 (4 GB) with the trained ensemble and, for reference, on an x86 cloud host. The Pi was not instrumented for energy, so every energy figure remains modelled from assumed fog-node wattages. Only model inference was timed; flow capture and feature extraction on the device were not.

Datasets. Edge-IIoTset, WUSTL-EHMS-2020 and the CICIDS2017 day were obtained from public mirrors whose row and class counts match the published descriptions; byte-level identity with the authors' releases could not be checked. IoTID20 figures come from the project's own earlier run on the original file. The detection-pipeline ablation, the explanation-stability measurements and the IoTID20 leakage table are single-partition results from the previous review, and their 20-partition replication is pending on the full pipeline.

Audit scope. The audit covers three papers selected because their confusion matrices are printed as numbers. That selection favours more carefully reported papers, so it is not a prevalence estimate, and we do not present it as one. Values were transcribed from publisher full text and reproduce the papers' printed metrics exactly, which guards against transcription error but not against an error the papers share with their own figures.

Human evaluation. The reader study is a pilot with students as the primary participants. Its results, once collected, speak to whether non-specialists can use the explanations, not to how clinicians or security operators would use them.

VIII. CONCLUSION

We presented a five-step framework for evaluating IoMT intrusion detection at the network level, from arithmetic audit to simulation, and applied it to three published systems and four public datasets. The audit confirmed metric-definition errors in one paper and a split drawn after oversampling in two, and showed experimentally that the ordering alone can add 0.29 to F1. Duplicate rates depend on the feature space retained, from under 1 % to 99 % on the same file. In an ns-3 fog network, rules with identical detection rates quarantined between 45 % and 100 % of legitimate wearables in the baseline, and the published rule all of them in every scenario; a congestion-coupled rule latched off under load, topology-driven concentration and a measured tail inference cost, and three mitigations each restored it in every run; and threshold adaptation proved bounded by the information it acts on, with a low-false-positive classifier gate moving that bound. On a Raspberry Pi 4 the trained ensemble reproduces its evaluated predictions exactly and costs 3.0 ms per window, enough to switch a congestion-coupled rule off at the Review 2 attack scale. Three items remain and are scripted: the reader study, the 20-partition IoTID20 tables and the public release.

REPRODUCIBILITY

The ns-3 simulator, the Review 2 program used for the regression check, every experiment definition, all raw runs (one line per run, written before aggregation), the dataset-leakage and multi-partition scripts, the audit extraction with source locations, the hardware harness, the reader-study materials and the manuscript build are archived on Zenodo, doi:10.5281/zenodo.23638403 (the DOI that always resolves to the latest version; the first release is 10.5281/zenodo.23638404). Datasets are not redistributed; their sources, checksums and row counts are listed with the code.

ACKNOWLEDGMENT

The authors thank the course faculty for the Review 3 roadmap, the maintainers of the datasets used here, and the authors of [1], [2] and [3], whose explicit reporting made the audit possible.

REFERENCES

[1] A. Berguiga, A. Harchay, and A. Massaoudi, "HIDS-IoMT: A deep learning-based intelligent intrusion detection system for the Internet of Medical Things," *IEEE Access*, vol. 13, pp. 32863-32882, 2025, doi: 10.1109/ACCESS.2025.3543127.

[2] Y. Hosain and M. Çakmak, "XAI-XGBoost: An innovative explainable intrusion detection approach for securing internet of medical things systems," *Sci. Rep.*, vol. 15, no. 1, Art. no. 22278, 2025, doi: 10.1038/s41598-025-07790-0.

[3] H. A. Tarish, R. Hassan, K. A. Zainol Ariffin, and M. M. Jaber, "Leakage-free evaluation of task-aware Nyström sampling for IoMT intrusion detection," *Front. Artif. Intell.*, vol. 9, 2026, doi: 10.3389/frai.2026.1957727.

[4] X. Liu, "Towards advanced AI-based solutions for securing IoMT in smart health information systems," *Sci. Rep.*, vol. 16, Art. no. 2079, 2026, doi: 10.1038/s41598-025-31850-0.

[5] J. A. Shaikh *et al.*, "RCLNet: An effective anomaly-based intrusion detection for securing the IoMT system," *Front. Digit. Health*, vol. 6, Art. no. 1467241, 2024, doi: 10.3389/fdgth.2024.1467241.

[6] A. Ghourabi and A. Alkhalil, "A federated learning model for detecting cyberattacks in Internet of Medical Things networks," *IEEE Access*, vol. 13, pp. 123018-123030, 2025, doi: 10.1109/ACCESS.2025.3588808.

[7] M. Alalhareth and S.-C. Hong, "Enhancing the Internet of Medical Things (IoMT) security with meta-learning: A performance-driven approach for ensemble intrusion detection systems," *Sensors*, vol. 24, no. 11, Art. no. 3519, 2024, doi: 10.3390/s24113519.

[8] G. Lazrek, K. Chetioui, Y. Balboul, S. Mazer, and M. El Bekkali, "An RFE/Ridge-ML/DL based anomaly intrusion detection approach for securing IoMT system," *Results Eng.*, vol. 23, Art. no. 102659, 2024, doi: 10.1016/j.rineng.2024.102659.

[9] E. Söğüt, M. Kayaoğlu, and O. Polat, "MSCA-Net: A multi-scale depthwise attention network for multi-class intrusion detection in Internet of Medical Things," *Sensors*, vol. 26, no. 13, Art. no. 4036, 2026, doi: 10.3390/s26134036.

[10] A. Mohammadi, M. Aminian, H. Ghahramani, and S. A. Asghari, "Securing healthcare with deep learning: A CNN-based model for medical IoT threat detection," in *Proc. 19th Iranian Conf. Intell. Syst. (ICIS)*, 2024, doi: 10.1109/ICIS64839.2024.10887510.

[11] S. Dadkhah, E. C. Pinto Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, "CICIoMT2024: A benchmark dataset for multi-protocol security assessment in IoMT," *Internet Things*, vol. 28, Art. no. 101351, 2024, doi: 10.1016/j.iot.2024.101351.

[12] V. Badiyani, "An intelligent intrusion detection system using machine learning on network flow features with an optimized stacking ensemble framework," manuscript, Vellore Institute of Technology, 2026.

[13] R. Jain *et al.*, "WUSTL EHMS 2020 dataset for Internet of Medical Things (IoMT) cybersecurity research," Washington Univ. in St. Louis. [Online]. Available: https://www.cse.wustl.edu/~jain/ehms/index.html

[14] I. Ullah and Q. H. Mahmoud, "A scheme for generating a dataset for anomalous activity detection in IoT networks," in *Proc. Can. Conf. Artif. Intell.*, Springer, 2020, pp. 508-520.

[15] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281-40306, 2022.

[16] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108-116.

[17] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3971-3988.

[18] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, 2021, pp. 7-12, doi: 10.1109/SPW53761.2021.00009.

[19] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, LNCS 13857, Springer, 2023, pp. 18-33, doi: 10.1007/978-3-031-31108-6_2.

[20] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, 2022, doi: 10.1109/CNS56114.2022.9947235.

[21] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE 9th Eur. Symp. Secur. Privacy (EuroS&P)*, 2024, pp. 658-675, doi: 10.1109/EuroSP60621.2024.00042.

[22] M. A. Bouke and A. Abdullah, "An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability," *Expert Syst. Appl.*, vol. 230, Art. no. 120715, 2023, doi: 10.1016/j.eswa.2023.120715.

[23] M. M. Galal, "Provenance, not behaviour: A serialisation artifact in Edge-IIoTset and a leakage-free benchmark for precision-agriculture intrusion detection," arXiv:2608.15761, 2026 (preprint).

[24] A. A. Alsulami, Q. Abu Al-Haija, A. Tayeb, and A. Alqahtani, "An intrusion detection and classification system for IoT traffic with improved data engineering," *Appl. Sci.*, vol. 12, no. 23, Art. no. 12336, 2022, doi: 10.3390/app122312336.

[25] P. Meyer, T. Häckel, T. Lübeck, F. Korf, and T. C. Schmidt, "A framework for the systematic assessment of anomaly detectors in time-sensitive automotive networks," in *Proc. IEEE Veh. Netw. Conf. (VNC)*, 2024, pp. 57-64, doi: 10.1109/VNC61989.2024.10576017.

[26] F. Rustam, I. Obaidat, and A. D. Jurcut, "MULTI-LF: A continuous learning framework for real-time malicious traffic detection in multi-environment networks," arXiv:2504.11575, 2025 (preprint).

[27] S. S. Chaudhari *et al.*, "Intelligent DDoS attack detection for IoT network scenario using NS3," in *Proc. Int. Conf. Integr. Intell. Commun. Syst. (ICIICS)*, 2023, pp. 1-6, doi: 10.1109/ICIICS59993.2023.10421286.

[28] R. M. Rajab, R. P. Ward, I. D. Wilson, and M. Abuhmida, "Assessing IoMT network resilience: A comparative performance analysis of simulated attacks using NS-3 [Appendix]," Zenodo, 2025, doi: 10.5281/zenodo.16810649.

[29] K. Hundman, V. Constantinou, C. Laporte, I. Colwell, and T. Soderstrom, "Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding," in *Proc. 24th ACM SIGKDD*, 2018, pp. 387-395, doi: 10.1145/3219819.3219845.

[30] Y. Yuan, Y. Huang, and Y. Wu, "Adaptive NAD: Online and self-adaptive unsupervised network anomaly detector," arXiv:2410.22967, 2024 (preprint).

[31] F. Ding, Y. Zhao, Z. Li, H. Tang, Y. Liu, and D. Guo, "Adaptive dynamic thresholds for unsupervised joint anomaly detection and trend prediction," *Sensors*, vol. 26, no. 1, Art. no. 257, 2026, doi: 10.3390/s26010257.

[32] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321-357, 2002.

[33] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765-4774.

[34] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 1135-1144.

[35] G. F. Riley and T. R. Henderson, "The ns-3 network simulator," in *Modeling and Tools for Network Simulation*, Springer, 2010, pp. 15-34.